SwiftPlan: Multimodal Embedding Alignment for High-Level Action Selection in Embodied Robots

Anonymous Author(s)
Affiliation
Address
email

Abstract: Embodied agents operating in real-world environments must integrate language understanding, visual perception, and high-level task decision-making. While recent vision-language models (VLMs) provide strong semantic representations, deploying them for robot action selection remains challenging due to fine-tuning cost, inference latency, and limited task-specific annotations. We propose SwiftPlan, a framework that formulates high-level robotic action selection as multimodal embedding-based prototype matching. Given the current observation, language instruction, and candidate action descriptions, SwiftPlan embeds all inputs into comparable representations and selects the next action by matching the fused observation-instruction representation to action embeddings. For long-horizon tasks, an external LLM provides sub-task instructions, while SwiftPlan performs embedding-based action selection for the active sub-task. Experiments on real-world robot video datasets and a closed-loop Isaac Sim environment show that SwiftPlan outperforms discriminative and generative baselines under limited supervision. The results further show that SwiftPlan supports closed-loop action selection for both short- and long-horizon tasks that require visual state understanding and language grounding.

Keywords: Robot Action Selection, Vision-Language Alignment, Multimodal Fusion

1 Introduction

Embodied robots operating in real-world environments must combine language understanding, visual perception, and task-level decision-making. Unlike static vision-language benchmarks, robotic action selection depends on the current scene: the robot must ground an instruction in visible objects, infer task-relevant states and affordances, and choose an action that is both semantically valid and physically executable. Recent LLM- and VLM-based systems have made substantial progress in robotic planning and control. High-level planners such as SayCan [1], Inner Monologue [2], and VoxPoser [3] use language models to decompose tasks and reason about action feasibility, while vision-language-action models such as PaLM-E [4], RT-2 [5], OpenVLA [6], and π_0, $\pi_{0.5}$ [7, 8] integrate visual and linguistic inputs into robot policies. These systems show the promise of large models for embodied decision-making, but they also highlight the cost of deploying them on domain-specific robots.

In practice, robot deployments often require fast adaptation to a specific scene, robot setup, or action space. Large generative planners often introduce task-specific tuning costs and multi-stage inference, where intermediate text, subgoals, or action traces are generated before an action is selected. These extra stages can increase latency and introduce additional failure points. In deployment, failures also tend to recur under specific layouts, viewpoints, or execution disturbances, suggesting the need for a decision module that can be updated from a small number of targeted samples. This motivates an action-selection framework that can adapt with limited supervision, incorporate targeted

Submitted to the 10th Conference on Robot Learning (CoRL 2026). Do not distribute.

correction data after deployment, and select high-level actions from the current visual-language context.

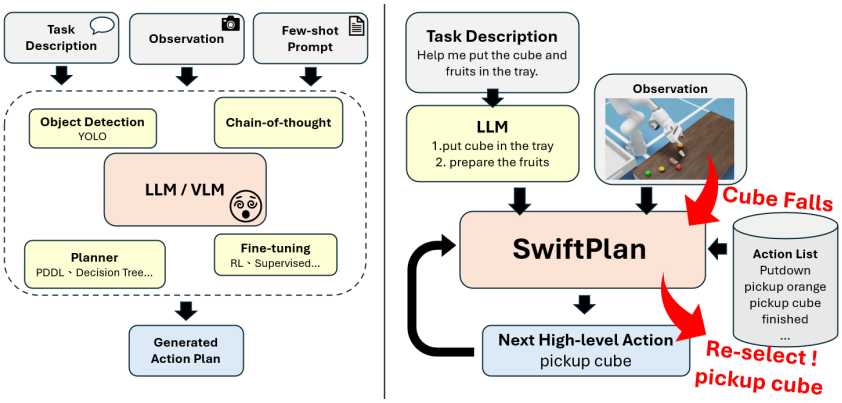

Figure 1: Conceptual comparison between conventional LLM/VLM-based planning pipelines and SwiftPlan's embedding-based action-selection module. For long-horizon tasks, an external LLM provides sub-task instructions, while SwiftPlan selects the next high-level action for the active sub-task from the current observation, instruction, and action list.

As shown in Figure 1, SwiftPlan formulates high-level action selection as prototype matching in a shared multimodal embedding space. For long-horizon tasks, sub-task instructions are provided by an external frozen LLM, while SwiftPlan focuses on the action-selection module after decomposition. Given the current observation, an active sub-task instruction, and a predefined action list, SwiftPlan encodes visual observations, language instructions, and candidate action descriptions into comparable representations before selecting the candidate action with the highest similarity. The model builds on SigLIP2 [9] and is trained in two stages. First, it performs BYOL-based self-supervised adaptation [10] on unlabeled scene images to improve visual representations for the target environment. Second, it learns multimodal action alignment with supervised contrastive learning [11]. For fusion, SwiftPlan uses cross-attention followed by a GMU-style gate [12], allowing the model to adjust the relative contribution of visual and language features according to the current task state.

We evaluate SwiftPlan on real-world robot video datasets and a closed-loop Isaac Sim environment. The results show that SwiftPlan improves action-selection accuracy over both discriminative and generative baselines, particularly under limited annotation. We further show that a small number of targeted post-deployment samples can improve the model on recurring failure cases.

2 Related Work

We review language-based planning approaches, vision-language models for robot decision-making, and multimodal alignment and fusion methods that motivate our action selection framework.

2.1 High-level Task Decision Making

High-level task decision-making requires converting natural-language instructions into executable task steps or action choices. LLM+P, LaMMA-P, EmboTeam, and UniDomain [13, 14, 15, 16] translate instructions into PDDL representations and use symbolic planners to derive action sequences. LLM-as-BT-Planner, LLM-BRAIn, and BTGenBot [17, 18, 19] instead generate Behavior Trees, which provide a structured and reactive representation for complex robot tasks. Code as Policies and PROGPROMPT [20, 21, 22] generate executable programs with logical conditions and API calls, allowing language models to express both task-level reasoning and low-level control logic.

Other systems combine language models with perception modules such as YOLO to ground instructions in detected objects before planning [23]. Gemini Robotics 1.5 and Hi Robot [24, 25] use vision-language reasoning to guide vision-language-action policies from the current observation.

Yell At Your Robot (YAY) [26] collects corrective language feedback during execution and uses it to improve high-level action prediction.

2.2 Multimodal Alignment and Fusion

Multimodal alignment aims to place inputs from different modalities in a shared representation space, so that semantically related image, text, or action features are close to each other. A common approach is to use dual encoders with contrastive objectives for cross-modal alignment. CLIP [27] and ALIGN [28] show that large-scale image-text supervision can produce representations with strong zero-shot transfer. SigLIP [29] replaces the softmax contrastive loss with a sigmoid loss, reducing the dependence on large batch sizes and improving training efficiency.

In embodied AI, several works learn reusable visual representations from human or robot demonstrations. R3M [30] combines time-contrastive learning with language alignment to learn visual representations useful for manipulation. Voltron [31] uses language-conditioned masked modeling to capture both semantic and physical information. LIV [32] embeds task progress in a multimodal space, so feature distances can be used as rewards for policy learning. CLIPort [33] combines CLIP features with Transporter-style spatial reasoning for language-conditioned pick-and-place.

Cross-modal decision-making also requires the model to condition visual features on the language instruction. FiLM [34] conditions visual features by predicting language-dependent scaling and shifting parameters. This method was subsequently applied to robotic control [35].

3 Approach

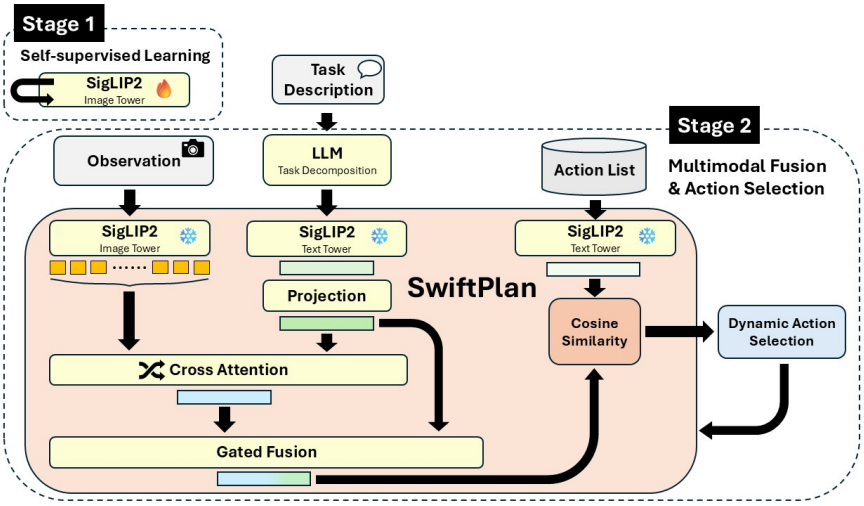

Figure 2: Overview of SwiftPlan. Stage 1 adapts the SigLIP2 image encoder to the target scene with BYOL-based self-supervised learning. Stage 2 uses sub-task instructions from an external LLM and fuses them with patch-level visual features through cross-attention and gated fusion. The resulting representation is matched to candidate action embeddings by cosine similarity.

SwiftPlan treats high-level action selection as matching the current visual-language state to candidate action descriptions in a shared embedding space. As shown in Figure 2, the framework first adapts the visual encoder with self-supervised learning and then trains a multimodal alignment module for action selection.

3.1 Stage 1: Self-supervised Fine-Tuning of the Visual Encoder

Stage 1 adapts the pretrained SigLIP2-base-patch16-512 visual encoder to the target environment before multimodal action training. In many robot settings, the scene layout and object categories are

relatively stable, but captured images still vary with lighting, sensor noise, viewpoint changes, and local appearance perturbations.

We use a BYOL-based objective to adapt the encoder without requiring action labels. Given an unlabeled image frame x, the teacher branch receives the clean view $x^{(t)} = x$, and the student branch receives an augmented view $x^{(s)} = \mathcal{A}_{\text{photo}}(x)$. The augmentation $\mathcal{A}_{\text{photo}}$ includes mild color jitter, blur, and small rotations, while avoiding cropping or strong geometric transformations that could disrupt patch-level object correspondence. Both views are passed through the SigLIP2 vision tower to obtain patch-token representations $Z_t, Z_s \in \mathbb{R}^{B\times N\times D}$. The student tokens are mapped by a projector g_{ϕ_s} and predictor q_ψ, yielding $P_s = q_\psi(g_{\phi_s}(Z_s))$. The teacher tokens are mapped only by the teacher projector, $\tilde{Z}_t = g_{\phi_t}(Z_t)$. The student branch is trained with a token-wise BYOL loss based on negative cosine similarity:

$$\mathcal{L}_{\text{BYOL}} = \frac{1}{BN}\sum_{b=1}^{B}\sum_{n=1}^{N}\left(2 - 2\cdot\frac{P_{s,b,n}^{\top}\tilde{Z}_{t,b,n}}{\|P_{s,b,n}\|_2\,\|\tilde{Z}_{t,b,n}\|_2}\right). \tag{1}$$

During this stage, most of the vision backbone is frozen; only the last Transformer blocks and LayerNorm parameters of the student encoder are fine-tuned, while the teacher branch is updated by exponential moving average.

3.2 Stage 2: Multimodal Feature Alignment and Fusion

Stage 2 trains the action-selection module on top of the scene-adapted visual encoder from Stage 1. The visual and text encoders are frozen, and we pre-extract the patch-level visual features $E_{\text{patch}} \in \mathbb{R}^{N\times D}$ and the instruction embedding $E_{\text{txt}} \in \mathbb{R}^{D}$ for each training sample. Candidate actions are also encoded as text prototypes $\{a_j\}_{j=1}^{C}$ using the frozen text encoder, where C is the number of action classes.

We project the instruction embedding and patch features into the same hidden space. To extract instruction-relevant visual information, the projected instruction feature h_{txt} is used as the query in a cross-attention layer, while the projected patch features serve as keys and values. This produces a language-conditioned visual feature h_{img} that summarizes the image regions most relevant to the current instruction.

The two modalities are then combined with a channel-wise gate. We concatenate h_{img} and h_{txt} and pass them through a small MLP to predict $g \in (0,1)^D$. The fused representation is computed as

$$h_{\text{fused}} = g \odot h_{\text{img}} + (1-g)\odot h_{\text{txt}}. \tag{2}$$

This gate lets the model adjust the contribution of visual and language features according to the current instruction and scene state.

For action selection, we normalize h_{fused} and the action prototypes $\{a_j\}$, then compute cosine similarities with a learnable scale parameter:

$$\ell_j = s\cdot\frac{h_{\text{fused}}^{\top}a_j}{\|h_{\text{fused}}\|_2\|a_j\|_2}, \tag{3}$$

where s is a learnable logit scale. The predicted action is the candidate with the highest logit.

The module is trained with a cross-entropy loss over the action labels. We also use supervised contrastive learning as an auxiliary objective to improve feature separation among action classes. For each anchor sample, same-class samples from the training set are used as positives. The final objective is

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda\,\mathcal{L}_{\text{SupCon}}, \tag{4}$$

where λ follows an exponential decay schedule during training. This places more emphasis on contrastive separation early in training and gradually shifts the objective toward action classification.

3.3 Task Decomposition via LLM

For long-horizon tasks, we use an external frozen LLM, Gemma-3-4B-IT, only as an upstream parser. The LLM decomposes a high-level task description $\mathcal{T}$ into an ordered sequence of sub-task instructions $\{s_1, s_2, \ldots, s_K\}$, and SwiftPlan is invoked separately for each sub-task.

For the closed-loop data, Action List includes task-completion labels such as *"put orange in the tray finished"*. When SwiftPlan selects the completion label for the current sub-task, the system marks that sub-task as completed and advances to the next instruction in the sequence. Otherwise, SwiftPlan continues selecting actions based on the current image and the active sub-task instruction.

Implementation details for Stage 1 and Stage 2 are provided in Appendix B.

4 Experimental Results

We evaluate SwiftPlan using main results and ablations, closed-loop multimodal task performance, and post-deployment correction with targeted failure samples.

We compare SwiftPlan with three high-level action-selection baselines. (1) **SigLIP2 + Projection + Cosine** uses pretrained SigLIP2 image and text encoders without scene-adaptive fine-tuning; mean-pooled visual features and instruction embeddings are projected and matched to fixed action-text prototypes by cosine similarity. (2) **YAY-style high-level policy** adapts the high-level language-prediction design of Yell At Your Robot (YAY), using the same SigLIP2 visual backbone to predict an action-language embedding and select the closest candidate action. (3) **Qwen3-VL-4B-Instruct with QLoRA fine-tuning** provides a generative VLM reference baseline. It is trained and evaluated with the same image-instruction-action format as SwiftPlan: given the current image, the same instruction field used by SwiftPlan, and the predefined candidate action set, the model predicts one valid action label. The prompt provides the valid action labels, object color cues, and task rules, and is shown in Appendix C.1.

Mean $\pm$ std is over five seeds with fixed test splits; Qwen3-VL-4B uses one run.

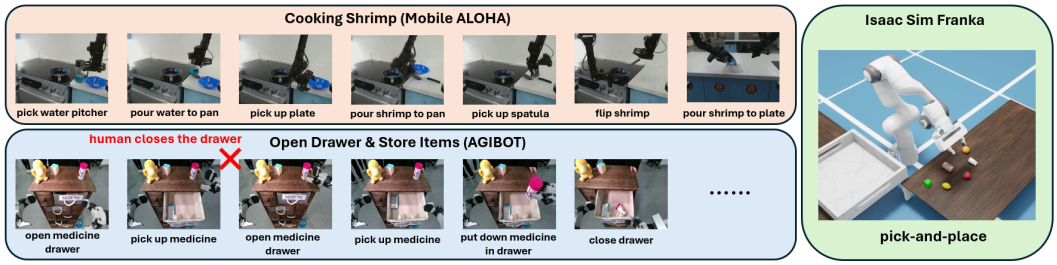

Figure 3: Representative evaluation datasets. The Mobile ALOHA subset contains sequential manipulation samples, the AGIBOT subset includes task trials with human interventions that change the scene state, and the Isaac Sim setup evaluates closed-loop short- and long-horizon pick-and-place tasks with state re-evaluation after each action.

4.1 Dataset and Tasks Setup

We evaluate SwiftPlan on real-world robot videos and a closed-loop simulated environment, as shown in Figure 3. For real-world evaluation, we use subsets of AGIBOT and Mobile ALOHA. AGIBOT contains task executions where human intervention changes the scene state, testing whether the model can update its action choice after the environment changes. Mobile ALOHA contains demonstrations from a moving robot viewpoint, introducing larger visual variation across frames. Since most existing robot video datasets provide raw demonstration trajectories or videos but do not include frame-level high-level action labels for next-action selection, we construct high-level action-selection samples from AGIBOT and Mobile ALOHA. Specifically, we uniformly sample

observations from each video and annotate the correct next high-level action under the corresponding task instruction. Each labeled sample consists of an observation, a task instruction, a candidate action list, and the next high-level action.

For closed-loop evaluation, we build an Isaac Sim pick-and-place environment with a Franka arm. Unlike the frame-level real-world evaluation, the simulator tests SwiftPlan during execution: after each selected action is executed, the robot observes the updated scene and selects the next action again. The tasks include both short-horizon instructions, such as *"put orange in the tray"*, and longer-horizon instructions, such as *"tidy up the properties"*. We also introduce execution disturbances, such as objects slipping after grasping, to evaluate whether the model can recover from the new scene state.

Detailed dataset statistics and action lists are provided in Appendix A. The training code, dataset annotations, and Isaac Sim evaluation data will be released upon publication.

4.2 Main Results and Ablation

For the main evaluation, the training and validation data consist of 31 AGIBOT videos and 27 Mobile ALOHA videos, yielding 648 and 518 annotated samples, respectively.

We further perform ablations to quantify the contribution of each component. Table 1 shows that SwiftPlan achieves the highest per-step action-selection accuracy on both test sets.

Table 1: Main results and ablation on disjoint test videos. For SwiftPlan variants, trainable parameters are reported as Stage 1 / Stage 2. The Concat-MLP Fusion variant replaces the learned gate with concat-MLP fusion over visual and language features.

Method	**Trainable Params (M)**	**AGIBOT Per-step Acc (%)**	**Mobile ALOHA Per-step Acc (%)**
Qwen3-VL-4B (QLoRA)	33	87.84	83.33
YAY	19.1	86.35 ± 1.34	90.21 ± 1.24
SigLIP2 + Proj. + Cosine	5.9	86.40 ± 0.97	87.89 ± 1.65
SwiftPlan w/o BYOL	0 / 5.9	89.23 ± 1.14	93.28 ± 0.93
SwiftPlan w/ Concat-MLP	59.9 / 5.9	89.64 ± 2.06	93.45 ± 1.74
SwiftPlan w/o $\mathcal{L}_{\text{SupCon}}$	59.9 / 5.9	90.00 ± 2.31	91.69 ± 1.16
SwiftPlan (ours)	59.9 / 5.9	$\mathbf{90.95 \pm 1.23}$	$\mathbf{93.57 \pm 1.26}$

The ablations indicate that the Stage 2 action-selection module accounts for most of the improvement. SwiftPlan w/o BYOL already outperforms the projection-only baseline, while BYOL provides a modest additional gain in the full model, especially on AGIBOT. The Concat-MLP and w/o $\mathcal{L}_{\text{SupCon}}$ variants both perform slightly worse than the full model, suggesting that both gated fusion and supervised contrastive learning contribute to the final model.

Mixed AGIBOT–Mobile ALOHA training results are provided in Appendix D.1, where SwiftPlan remains stable with a larger candidate action set. Random-action and task-majority reference baselines are reported in Appendix C.2.

4.3 Closed-loop Multimodal Task Performance

In Isaac Sim, we evaluate SwiftPlan on 27 held-out closed-loop pick-and-place tasks, including both short- and long-horizon instructions. Unlike the real-world video setting, where frames are labeled independently, the simulator evaluates the model during execution. After each selected high-level action is executed, the robot receives a new observation, and the model is queried again with the updated scene and the same task instruction. Thus, the model must choose actions from the current execution state, rather than from isolated frames.

Each task contains multiple manipulable objects, so the correct action depends on both the current scene and the active instruction.

Table 2: Closed-loop multimodal task performance in Isaac Sim on held-out test samples. A task is counted as successful only when all required high-level actions are selected correctly.

Method	Per-step Acc (%)	Task Success Rate (%)
Qwen3-VL-4B (QLoRA)	61.33	7.40
YAY	61.60 ± 3.42	2.96 ± 3.10
SigLIP2 + Proj. + Cosine	84.53 ± 1.97	52.59 ± 6.09
SwiftPlan w/ Concat-MLP Fusion	91.87 ± 2.64	70.37 ± 9.44
SwiftPlan (ours)	**93.47 ± 1.28**	**74.81 ± 6.17**

Table 2 shows that SwiftPlan achieves 93.47% per-step accuracy and a 74.81% task success rate. In the Qwen3-VL-4B rollouts, failures often occur near task completion: even when the target objects are already in the tray and the gripper is empty, the model may still predict *pickup* or *putdown*, preventing the controller from triggering the completion label and advancing to the next sub-task. Compared with SigLIP2 + Projection + Cosine, SwiftPlan improves per-step accuracy by 8.94 percentage points and task success by 22.22 percentage points. This suggests that text-guided patch attention and gated fusion improve state-aware action selection.

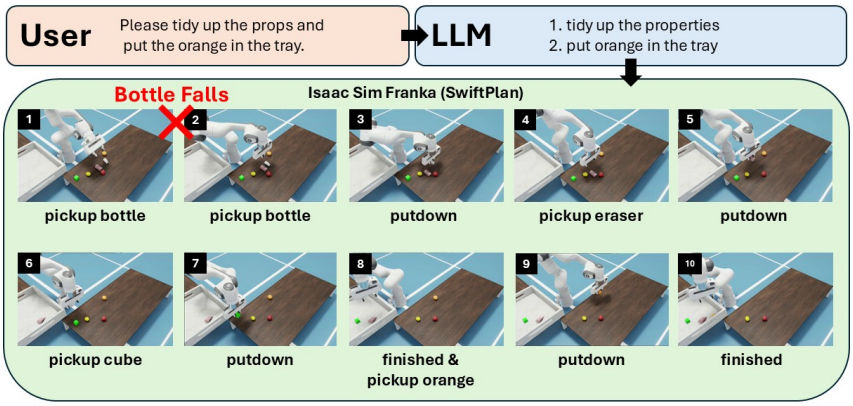

Figure 4: Qualitative closed-loop rollout in Isaac Sim. The LLM decomposes the instruction into two sub-tasks. SwiftPlan executes the first sub-task, recovers from a bottle-fall disturbance, predicts the completion label, and proceeds to the second sub-task.

Figure 4 illustrates how SwiftPlan updates its decision from the current scene rather than following a fixed open-loop sequence. After the bottle is displaced during execution, SwiftPlan re-evaluates the updated observation, selects *"pickup bottle"* again, and advances to the next sub-task only after predicting the corresponding completion label.

Appendix D.2 provides a channel-wise gate analysis, where state-dependent actions such as *putdown* show a higher ratio of visual-dominant channels.

Table 3 reports representative per-step inference latency for the generative VLM baseline and SwiftPlan. Full latency comparisons are provided in Appendix E.

Table 3: Representative per-step inference latency measured on a single NVIDIA RTX A5000 GPU.

Method	Input	Latency / Step
Qwen3-VL-4B (QLoRA)	Image + Instruction	∼900 ms
SwiftPlan (ours)	Image + Instruction	∼34 ms

4.4 Offline Post-deployment Correction

We further test whether SwiftPlan can be corrected after deployment using a small number of targeted failure samples. Using the Isaac Sim split from Section 4.3, we first identify test cases that are

misclassified by both SwiftPlan and the SigLIP2 + Projection + Cosine baseline. The two methods share five failure cases, indicating scene configurations that remain difficult even after multimodal fusion.

For each shared failure case, we re-run the robot in Isaac Sim and collect seven additional observations under similar visual conditions, resulting in 35 newly annotated samples. These samples are added to the original training set, and Stage 2 is retrained before redeployment. This is an offline correction procedure rather than online test-time adaptation; it evaluates whether the action-selection module can be updated from a small number of deployment-time failure examples.

Table 4: Offline post-deployment correction results on the held-out Isaac Sim test set. Per-step accuracy measures action selection at each decision step, while task success requires all high-level decisions in a closed-loop task to be correct.

SwiftPlan Setting	**Per-step Acc (%)**	**Task Success Rate (%)**
Before correction	93.47 ± 1.28	74.81 ± 6.17
After correction	$\mathbf{95.84 \pm 1.37}$	$\mathbf{81.48 \pm 5.24}$

Table 4 shows that adding 35 targeted failure samples improves per-step accuracy from 93.47% to 95.84% and task success from 74.81% to 81.48%. These results suggest that SwiftPlan can benefit from small-scale offline correction after deployment.

5 Conclusion

We presented SwiftPlan, a framework for high-level action selection in embodied robots. It formulates action selection as multimodal embedding matching: after task decomposition, the current observation and active sub-task instruction are fused and matched to candidate action descriptions. With scene-adapted visual features, text-guided patch attention, and gated fusion, SwiftPlan selects actions from the current visual-language state without generating intermediate action traces at each step. Experiments on real-world robot videos and closed-loop Isaac Sim tasks show that SwiftPlan improves over discriminative and generative baselines under limited supervision, supports state-aware re-selection during execution, and can be updated with a small number of targeted failure samples.

6 Limitations and Future Work

SwiftPlan focuses on high-level action selection rather than end-to-end low-level control. It selects from a predefined action list and cannot produce actions outside the provided label set. This keeps inference lightweight, but limits open-world use when new objects, skills, or action types appear. For long-horizon tasks, SwiftPlan also relies on an external LLM for task decomposition, so decomposition errors remain outside the action-selection model.

Our real-world evaluation uses AGIBOT and Mobile ALOHA videos for offline frame-level action prediction rather than closed-loop execution on physical robots. Isaac Sim provides closed-loop evaluation, but remains a controlled pick-and-place environment and does not capture all sim-to-real factors, including domain shift, sensing noise, calibration errors, contact dynamics, and changing layouts. Still, it uses the same visual-observation, task-instruction, and action-list interface as the real-world video experiments, and SwiftPlan does not use simulator privileged state. In physical deployment, the selected high-level action labels would be mapped to robot-specific low-level controllers or skills while preserving this interface. Thus, the closed-loop results suggest that the proposed high-level decision interface is compatible with physical deployment, while full closed-loop evaluation on physical robots remains future work.

References

[1] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.

[2] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, et al. Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv:2207.05608*, 2022.

[3] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei. Voxposer: Composable 3d value maps for robotic manipulation with language models. *arXiv preprint arXiv:2307.05973*, 2023.

[4] D. Driess, F. Xia, M. S. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, et al. Palm-e: An embodied multimodal language model. *arXiv preprint arXiv:2303.03378*, 2023.

[5] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.

[6] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[7] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[8] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.

[9] M. Tschannen, A. Gritsenko, X. Wang, M. F. Naeem, I. Alabdulmohsin, N. Parthasarathy, T. Evans, L. Beyer, Y. Xia, B. Mustafa, O. Hénaff, J. Harmsen, A. Steiner, and X. Zhai. Siglip 2: Multilingual vision-language encoders with improved semantic understanding, localization, and dense features, 2025. URL https://arxiv.org/abs/2502.14786.

[10] J.-B. Grill, F. Strub, F. Altché, C. Tallec, P. Richemond, E. Buchatskaya, C. Doersch, B. Avila Pires, Z. Guo, M. Gheshlaghi Azar, et al. Bootstrap your own latent-a new approach to self-supervised learning. *Advances in neural information processing systems*, 33: 21271–21284, 2020.

[11] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, and D. Krishnan. Supervised contrastive learning. *Advances in neural information processing systems*, 33:18661–18673, 2020.

[12] J. Arevalo, T. Solorio, M. Montes-y Gómez, and F. A. González. Gated multimodal units for information fusion. *arXiv preprint arXiv:1702.01992*, 2017.

[13] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, and P. Stone. Llm+ p: Empowering large language models with optimal planning proficiency. *arXiv preprint arXiv:2304.11477*, 2023.

[14] X. Zhang, H. Qin, F. Wang, Y. Dong, and J. Li. Lamma-p: Generalizable multi-agent long-horizon task allocation and planning with lm-driven pddl planner. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 10221–10221. IEEE, 2025.

[15] H. Zeng and P. Li. H-aim: Orchestrating llms, pddl, and behavior trees for hierarchical multi-robot planning. *arXiv preprint arXiv:2601.11063*, 2026.

[16] H. Ye, Y. Xiao, C. Lu, and P. Cai. Unidomain: Pretraining a unified pddl domain from real-world demonstrations for generalizable robot task planning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

[17] J. Ao, F. Wu, Y. Wu, A. Swiki, and S. Haddadin. Llm-as-bt-planner: Leveraging llms for behavior tree generation in robot task planning. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1233–1239. IEEE, 2025.

[18] A. Lykov and D. Tsetserukou. Llm-brain: Ai-driven fast generation of robot behaviour tree based on large language model. In *2024 2nd International Conference on Foundation and Large Language Models (FLLM)*, pages 392–397. IEEE, 2024.

[19] R. A. Izzo, G. Bardaro, and M. Matteucci. Btgenbot: Behavior tree generation for robotic tasks with lightweight llms. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 9684–9690. IEEE, 2024.

[20] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng. Code as policies: Language model programs for embodied control. In *2023 IEEE International conference on robotics and automation (ICRA)*, pages 9493–9500. IEEE, 2023.

[21] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg. Progprompt: Generating situated robot task plans using large language models. *arXiv preprint arXiv:2209.11302*, 2022.

[22] H. Luo, J. Wu, J. Liu, and M. F. Antwi-Afari. Large language model-based code generation for the control of construction assembly robots: A hierarchical generation approach. *Developments in the Built Environment*, 19:100488, 2024.

[23] H. Liu, Y. Zhu, K. Kato, I. Kondo, T. Aoyama, and Y. Hasegawa. Llm-based human-robot collaboration framework for manipulation tasks. *arXiv preprint arXiv:2308.14972*, 2023.

[24] G. R. Team, A. Abdolmaleki, S. Abeyruwan, J. Ainslie, J.-B. Alayrac, M. G. Arenas, A. Balakrishna, N. Batchelor, A. Bewley, J. Bingham, et al. Gemini robotics 1.5: Pushing the frontier of generalist robots with advanced embodied reasoning, thinking, and motion transfer. *arXiv preprint arXiv:2510.03342*, 2025.

[25] L. X. Shi, B. Ichter, M. Equi, L. Ke, K. Pertsch, Q. Vuong, J. Tanner, A. Walling, H. Wang, N. Fusai, et al. Hi robot: Open-ended instruction following with hierarchical vision-language-action models. *arXiv preprint arXiv:2502.19417*, 2025.

[26] L. X. Shi, Z. Hu, T. Z. Zhao, A. Sharma, K. Pertsch, J. Luo, S. Levine, and C. Finn. Yell at your robot: Improving on-the-fly from language corrections. *arXiv preprint arXiv:2403.12910*, 2024.

[27] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PmLR, 2021.

[28] C. Jia, Y. Yang, Y. Xia, Y.-T. Chen, Z. Parekh, H. Pham, Q. Le, Y.-H. Sung, Z. Li, and T. Duerig. Scaling up visual and vision-language representation learning with noisy text supervision. In *International conference on machine learning*, pages 4904–4916. PMLR, 2021.

[29] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer. Sigmoid loss for language image pre-training. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 11975–11986, October 2023.

[30] S. Nair, A. Rajeswaran, V. Kumar, C. Finn, and A. Gupta. R3m: A universal visual representation for robot manipulation. *arXiv preprint arXiv:2203.12601*, 2022.

[31] S. Karamcheti, S. Nair, A. S. Chen, T. Kollar, C. Finn, D. Sadigh, and P. Liang. Language-driven representation learning for robotics. *arXiv preprint arXiv:2302.12766*, 2023.

[32] Y. J. Ma, V. Kumar, A. Zhang, O. Bastani, and D. Jayaraman. Liv: Language-image representations and rewards for robotic control. In *International Conference on Machine Learning*, pages 23301–23320. PMLR, 2023.

[33] M. Shridhar, L. Manuelli, and D. Fox. Cliport: What and where pathways for robotic manipulation. In *Conference on robot learning*, pages 894–906. PMLR, 2022.

[34] E. Perez, F. Strub, H. De Vries, V. Dumoulin, and A. Courville. Film: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.

[35] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.

A Dataset Details

A.1 Dataset Statistics

Table 5 summarizes the number of videos, closed-loop tasks, and annotated samples used in each dataset split. For AGIBOT and Mobile ALOHA, the data are split at the video level. For Isaac Sim, the data are collected as closed-loop execution samples, and the held-out test set is grouped into closed-loop tasks.

Table 5: Dataset statistics used in the experiments. For AGIBOT and Mobile ALOHA, "videos/-tasks" denotes the number of robot demonstration videos. For Isaac Sim, the test split is grouped into closed-loop tasks rather than videos.

Dataset	Split	Videos / Tasks	Samples	Usage
AGIBOT	Train + Val	31 videos	648	Main evaluation training and validation
AGIBOT	Test	23 videos	444	Held-out video-level test set
Mobile ALOHA	Train + Val	27 videos	518	Main evaluation training and validation
Mobile ALOHA	Test	18 videos	342	Held-out video-level test set
Isaac Sim	Train + Val	–	248	Closed-loop action-selection training and validation
Isaac Sim	Test	27 tasks	150	Held-out closed-loop evaluation
Post-deployment	Adaptation	5 failure cases	35	Additional targeted samples for replay-style adaptation

Note. For all Train + Val splits, 80% of the data is used for training and 20% is used for validation. The split is generated with random seed 42. Test data are kept disjoint and are not used in either training stage.

A.2 Action Lists

A.2.1 AGIBOT Action List

Table 6 lists the high-level action labels used for the AGIBOT subset. The AGIBOT action list contains 23 unique action labels across five task instructions.

Table 6: High-level action list for the AGIBOT subset. Actions are grouped by task instruction.

Task Instruction	High-level Action Labels	# Actions
Put the can and the paper ball into the trash can.	*pickup can*; *putdown can in trash can*; *pickup paper ball*; *putdown paper ball in trash can*	4
Put the detergent bottles into the cardboard box.	*pickup detergent*; *putdown detergent in box*	2
Put the medicine bottle into the medicine drawer and the toys into the toy drawer.	*open medicine drawer*; *pickup medicine*; *putdown medicine in drawer*; *open toy drawer*; *pickup duck toy*; *putdown duck toy in drawer*; *pickup magic cube*; *putdown magic cube in drawer*; *close drawer*	9
Put the pink bowl into the oven and close the door.	*open oven*; *pickup plate*; *putdown plate in oven*; *push button*; *close oven*	5
Pick up the red stamp and press it onto the paper on the clipboard.	*pickup stamp*; *press stamp onto paper*; *putdown stamp*	3

A.2.2 Mobile ALOHA Action List

Table 7 lists the high-level action labels used for the Mobile ALOHA subset. The Mobile ALOHA action list contains 22 unique action labels across six task instructions. Actions are grouped by task

instruction for readability; shared actions such as *"open cabinet"* and *"close cabinet"* may appear in multiple task groups.

Table 7: High-level action list for the Mobile ALOHA subset. Actions are grouped by task instruction.

Task Instruction	**High-level Action Labels**	**# Actions**
Polish the wine glass.	*pickup cloth*; *pickup wine glass*; *place wine glass on cloth*	3
Wash the pan.	*pickup pan*; *move pan to sink*; *open faucet*; *rotate pan*	4
Cook the shrimp.	*pickup water pitcher*; *pour water to pan*; *pickup plate*; *pour shrimp to pan*; *pickup spatula*; *flip shrimp*; *pickup pan*; *pour shrimp to plate*	8
Place the pot inside the cabinet.	*open cabinet*; *pickup pot*; *putdown pot in cabinet*; *close cabinet*	4
Place the metal basin inside the cabinet.	*open cabinet*; *pickup metal basin*; *putdown metal basin in cabinet*; *close cabinet*	4
Place the foil tray inside the cabinet.	*open cabinet*; *pickup foil tray*; *putdown foil tray in cabinet*; *close cabinet*	4

A.2.3 Isaac Sim Action List

Table 8 lists the high-level action labels used for the Isaac Sim closed-loop environment. The Isaac Sim action list contains 13 unique action labels across six task instructions. The action *"putdown"* is shared across tasks and represents placing the currently held object into the target tray.

Table 8: High-level action list for the Isaac Sim environment. Actions are grouped by task instruction.

Task Instruction	**High-level Action Labels**	**# Actions**
prepare the fruits	*pickup orange*; *putdown*; *pickup strawberry*; *pickup lemon*; *prepare the fruits finished*	5
put orange in the tray	*pickup orange*; *putdown*; *put orange in the tray finished*	3
put lemon in the tray	*pickup lemon*; *putdown*; *put lemon in the tray finished*	3
put cube in the tray	*pickup cube*; *putdown*; *put cube in the tray finished*	3
put eraser in the tray	*pickup eraser*; *putdown*; *put eraser in the tray finished*	3
tidy up the properties	*pickup bottle*; *putdown*; *pickup eraser*; *pickup cube*; *tidy up the properties finished*	5

B Implementation Details

B.1 BYOL Self-supervised Adaptation

Stage 1 adapts the SigLIP2 visual encoder using unlabeled frames from the training and validation videos only. The teacher branch receives the clean image, while the student branch receives a lightly augmented view of the same frame. The BYOL loss is computed at the patch-token level.

Table 9: Implementation details for Stage 1 BYOL self-supervised adaptation.

Item	**Setting**
Backbone	SigLIP2-base-patch16-512
Training samples per epoch	10,000 randomly sampled frames
Validation samples	500 randomly sampled frames
Epochs	15
Batch size	4
Gradient accumulation	8
Effective batch size	32
Optimizer	AdamW
Vision encoder learning rate	1×10^{-5}
Projector / predictor learning rate	1×10^{-4}
Weight decay	1×10^{-2}
Gradient clipping	1.0
EMA momentum	0.996
Trainable vision parameters	Last 8 ViT blocks and LayerNorm parameters
Projector hidden dimension	2048
Projector output dimension	256

B.2 Multimodal Alignment Training

Stage 2 trains the multimodal action-selection module using annotated high-level action labels. We first pre-extract visual patch features, instruction embeddings, and action text prototypes using the Stage 1 adapted SigLIP2 model. During Stage 2 training, the visual encoder and text encoder are kept frozen, and only the action-selection alignment module is optimized.

Table 10: Implementation details for Stage 2 multimodal alignment and action-selection training.

Item	**Setting**
Visual backbone	Stage 1 adapted SigLIP2-base-patch16-512
Text encoder	Frozen SigLIP2 text encoder
Visual features	Pre-extracted patch-level hidden states
Instruction features	Frozen text embeddings of task instructions
Action prototypes	Frozen text embeddings of candidate action labels
Maximum text length	64 tokens
Embedding batch size	8
Training batch size	16
Epochs	40
Optimizer	AdamW
Learning rate	1×10^{-4}
Weight decay	1×10^{-2}
Learning rate schedule	Cosine decay with 5% warmup
Gradient clipping	1.0
Cross-attention heads	8
Dropout	0.1
Initial temperature	0.07
Trainable modules	Text projection, image projection, cross-attention, gated fusion, fusion refinement, and logit scale
Frozen modules	Visual encoder, text encoder, and action text prototypes
Training objective	Cross-entropy loss with supervised contrastive auxiliary loss
SupCon temperature	0.07
SupCon positive samples	2 same-class samples from the training pool
SupCon views	3 views: anchor + 2 positives
SupCon weight schedule	Exponential decay from 1.0 to 0.1
Checkpoint selection	Best validation Top-1 accuracy, with validation Top-3 accuracy as tie-break

C Baseline Details

C.1 VLM Baseline Prompt

For the Qwen3-VL-4B QLoRA baseline, we use a single image-conditioned prompt for high-level action selection. The prompt provides the current task instruction, object color cues, task rules, and the complete valid action label set. The program does not filter candidate actions by task or provide additional state information; the VLM must infer the current robot state from the image and instruction.

Listing 1: Prompt used for the Qwen3-VL-4B QLoRA baseline in the Isaac Sim action-selection experiment.

```
You are a robot high-level action selection model.

Given the current image and task instruction, predict the robot arm's next
    high-level action.

CURRENT TASK:
"{instruction}"

The program will not filter actions or decide any condition for you.
You must infer the current robot state from the image.

OBJECT COLOR CUES:
bottle = white
eraser = pink
cube = green
orange = orange
lemon = yellow
strawberry = red

IMPORTANT:
The task may already be in the middle or at the end.
Do not assume the robot is at the first step.
Do not choose an action only from the task name.
Use the image to infer the current state.

INTERNAL STATE CHECK:
Before choosing the action, silently decide which state is shown in the image:

A. HOLDING_STATE:
The gripper is holding, grasping, lifting, or enclosing an object.
If this is true, the next action must be:
putdown

B. COMPLETE_STATE:
The gripper is empty, and all target objects for the current task are already
    inside the tray.
No target object for the current task remains on the table.
If this is true, output the finished action for the current task.

C. PICKUP_STATE:
The gripper is empty, the task is not complete, and at least one required
    target object is still on the table.
Pick the first remaining required object in the current task order.

STATE PRIORITY:
1. HOLDING_STATE has highest priority.
2. COMPLETE_STATE is checked only if the gripper is empty.
3. PICKUP_STATE is used only if the gripper is empty and the task is not
    complete.

TASK RULES:
```

```
prepare the fruits:
required order = orange -> strawberry -> lemon
finished action = prepare the fruits finished

put orange in the tray:
required order = orange
finished action = put orange in the tray finished

put lemon in the tray:
required order = lemon
finished action = put lemon in the tray finished

put cube in the tray:
required order = cube
finished action = put cube in the tray finished

put eraser in the tray:
required order = eraser
finished action = put eraser in the tray finished

tidy up the properties:
required order = bottle -> eraser -> cube
finished action = tidy up the properties finished

ACTION SELECTION RULES:
- If the gripper is holding any object, output exactly: putdown
- If all required objects are already in the tray and the gripper is empty,
    output the current task's finished action.
- If the gripper is empty and the task is not complete, output pickup for the
    first required object that is still on the table.
- Do not pick an object that is already inside the tray.
- Do not output a finished action if the gripper is holding an object.
- Do not output a pickup action if the gripper is holding an object.

VALID ACTION LABELS:
pickup orange
pickup strawberry
pickup lemon
putdown
prepare the fruits finished
put orange in the tray finished
put lemon in the tray finished
pickup cube
put cube in the tray finished
pickup eraser
put eraser in the tray finished
pickup bottle
tidy up the properties finished

CURRENT TASK:
"{instruction}"

Return only valid JSON:
{
  "action_label": "<one valid action label>"
}
```

C.2 Reference Baselines for Action-selection Difficulty

To contextualize the difficulty of the action-selection problem, we report two simple reference baselines. *Random Action* uniformly samples from the valid candidate action list of each task, rather than from the global action vocabulary. *Task Majority* predicts the most frequent action label for

each task instruction. These baselines measure the effect of small task-specific action spaces and label-frequency bias.

Table 11: Reference baselines for action-selection difficulty. Random Action samples uniformly from the valid candidate action list for each task, while Task Majority predicts the most frequent action for each task instruction.

Method	AGIBOT Per-step Acc (%)	Mobile ALOHA Per-step Acc (%)	Isaac Sim Per-step Acc (%)
Random Action	27.56	21.25	25.07
Task Majority	34.23	35.38	37.33

D Additional Experimental Results

D.1 Mixed-dataset Training with a Larger Action Set

We further evaluate whether SwiftPlan remains stable when the action space becomes larger and the training data come from multiple robot datasets. Instead of training separate models on AGIBOT and Mobile ALOHA, we train a single SwiftPlan model using the combined training and validation splits from both datasets. The model uses the same architecture and training objective as in the main experiments, but candidate action prototypes are constructed from the union of the AGIBOT and Mobile ALOHA action labels. We then evaluate the same model on the original held-out AGIBOT and Mobile ALOHA test videos.

This setting is more challenging than dataset-specific training because the model must distinguish actions from a larger candidate set while handling visual observations from two different robot data sources. As shown in Table 12, joint training does not lead to a clear performance drop on either test set. The result provides initial evidence that SwiftPlan remains robust when the predefined action set is enlarged, without sacrificing accuracy on the original evaluation tasks.

Table 12: Mixed-dataset training with a larger candidate action set. Separate training denotes the main SwiftPlan setting, where AGIBOT and Mobile ALOHA are trained with dataset-specific action lists. Mixed training uses a single SwiftPlan model trained on the combined AGIBOT and Mobile ALOHA training data, with candidate action prototypes constructed from the union of both action lists, and evaluated on the original held-out test videos.

Training Setting	Training Data	AGIBOT Test Acc (%)	Mobile ALOHA Test Acc (%)
Separate training	Dataset-specific	90.95 ± 1.23	93.57 ± 1.26
Mixed training	AGIBOT + Mobile ALOHA	91.17 ± 1.11	93.51 ± 1.26

D.2 Gated Fusion Analysis

To inspect how the gate changes with the current robot state, we report representative channel-wise gate statistics in Table 13. Channels with $g > 0.6$ are treated as visual-dominant, while those with $g < 0.4$ are treated as text-dominant. The gate changes across action types: *"putdown"* has the highest visual-dominant ratio, consistent with its dependence on the gripper state, whereas object-selection and completion actions remain more balanced between visual and language features.

Table 13: Representative gate statistics under the instruction *"prepare the fruits"*.

Action	Gate Mean	Visual-dominant Ratio	Text-dominant Ratio
pickup orange	0.5016	26.82%	27.86%
putdown	0.5625	46.22%	21.61%
prepare the fruits finished	0.4853	25.26%	29.17%

E Inference Latency

We report representative per-step inference latency for action selection. All measurements are taken on the same workstation with a single NVIDIA RTX A5000 GPU. The timing measures one action-selection step after model loading, including image and instruction encoding when applicable.

Table 14: Representative inference latency per action-selection step measured on the same workstation with a single NVIDIA RTX A5000 GPU.

Method	**Input**	**Latency / Step**
Qwen3-VL-4B (QLoRA)	Image + Instruction	$\sim$900 ms
YAY	Image	$\sim$36 ms
SigLIP2 + Proj. + Cosine	Image + Instruction	$\sim$34 ms
SwiftPlan (ours)	Image + Instruction	$\sim$34 ms